

School of Computer Science and Artificial Intelligence

Lab Assignment # 11.1

Program	: B. Tech (CSE)
Specialization	: CSE
Course Title	: AI Assisted coding
Course Code	: 23CS002PC304
Semester	: II
Academic Session	: 2025-2026
Name of Student	: A VamshiKrishna
Enrollment No.	: 2403A51L47
Batch No.	: 52
Date	: 16-02-2026

Task 1 - Stack Implementation:

Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Prompt: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

```
class Stack:

    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.items.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0

obj = Stack()
obj.push(1)
obj.push(2)
print(obj.peek())
print(obj.pop())
print(obj.is_empty())
print(obj.pop())
```

```
2
2
False
1
```

Explanations:

- The Stack class implements a LIFO (Last In, First Out) data structure.
- The methods push, pop, peek, and is_empty are used to manipulate and inspect the stack.

Task 2 - Queue Implementation:

Use AI to implement a Queue using Python lists.

Prompt:

Implement a Queue data structure in Python using lists.

The Queue should follow the FIFO (First In, First Out) principle.

```
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        self.items.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("Dequeue from an empty queue")

    def peek(self):
        if not self.is_empty():
            return self.items[0]
        else:
            raise IndexError("Peek from an empty queue")

    def size(self):
        return len(self.items)

    def is_empty(self):
        return len(self.items) == 0

# Example usage:
if __name__ == "__main__":
    queue = Queue()
    queue.enqueue(1)
    queue.enqueue(2)
    queue.enqueue(3)

    print(queue.peek()) # Output: 1
    print(queue.dequeue()) # Output: 1
    print(queue.size()) # Output: 2
    print(queue.is_empty()) # Output: False
```

Explanation:

- The Queue class implements a first-in-first-out (FIFO) data structure.
- It supports operations like enqueue (add item), dequeue (remove item), peek (view first item), and size (get number of items).

Task 3 – Linked List:

Use AI to generate a Singly Linked List with insert and display methods.

Prompt: Generate a Singly Linked List with insert and display methods.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()

# Example usage:
linked_list = LinkedList()
linked_list.insert(10)
linked_list.insert(20)
linked_list.insert(30)
linked_list.display() # Output: 10 20 30
```

```
C:\Users\atur\OneDrive\Desktop\Ai_as
10 20 30
```

Explanation:

- The linked list has 3 nodes with data 10, 20, and 30.
- Each node points to the next node in the sequence:
- 10 -> 20 -> 30 -> None

Task 4: Use AI to create a BST with insert and in-order traversal methods

Prompt: Create a Binary Search Tree (BST) implementation in Python with recursive insert and in-order traversal methods.

```
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key

class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_recursively(self.root, key)

    def _insert_recursively(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = Node(key)
            else:
                self._insert_recursively(node.left, key)
        else:
            if node.right is None:
                node.right = Node(key)
            else:
                self._insert_recursively(node.right, key)

    def inorder_traversal(self):
        return self._inorder_recursively(self.root)

    def _inorder_recursively(self, node):
        res = []
        if node:
            res = self._inorder_recursively(node.left)
            res.append(node.val)
```

```
        res.append(node.val)
        res = res + self._inorder_recursively(node.right)
    return res
```

```
# Example usage:
```

```
if __name__ == "__main__":
```

```
    bst = BST()
```

```
    bst.insert(5)
```

```
    bst.insert(3)
```

```
    bst.insert(7)
```

```
    bst.insert(2)
```

```
    bst.insert(4)
```

```
    bst.insert(6)
```

```
    bst.insert(8)
```

```
    print("In-order Traversal:", bst.inorder_traversal())
```

Explanation :

- The BST class uses a Node class to represent each element in the tree.
- The insert method recursively places each new value in the correct position in the tree, and inorder_traversal returns a sorted list of values.

Task 5: Task: Use AI to implement a hash table with basic insert, search, and delete methods

Prompt: Implement a hash table with basic insert, search, and delete methods, using chaining for collision

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return
        self.table[index].append((key, value))

    def search(self, key):
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None

    def delete(self, key):
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return True
        return False
```

```
if __name__ == "__main__":  
    ht = HashTable()  
    ht.insert("name", "Alice")  
    ht.insert("age", 30)  
    print(ht.search("name"))  
    print(ht.search("age"))  
    ht.delete("name")  
    print(ht.search("name"))  
    print(ht.search("age"))
```

```
C:\Users\aelur\OneDrive\Desktop\Ai_assistanr>C:/Python314/python  
Alice  
30  
None  
30
```

Explanation:

- The hash table uses chaining to handle collisions.
- Each bucket in the table is a list that stores key-value pairs.

Task 6 -Graph Representation:

Use AI to implement a graph using an adjacency list.

Prompt: Implement a graph using an adjacency list in Python. The graph should have methods to add vertices, add edges, and display connections.

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)

    def display_connections(self):
        for vertex, edges in self.graph.items():
            print(f"{vertex}: {' '.join(edges)}")

# Example usage:
g = Graph()
g.add_vertex("A")
g.add_vertex("B")
g.add_vertex("C")
g.add_edge("A", "B")
g.add_edge("B", "C")
g.display_connections()
```

Explanation:

- The Graph class uses a dictionary to represent the graph as an adjacency list, allowing for efficient storage and retrieval of vertices and their connections.
- The methods provided allow for adding vertices, creating edges between them, and displaying the connections in a readable format.

Task 7: Use AI to implement a priority queue using Python's heapq module.

Prompt:

- Implement a priority queue using Python's heapq module. The priority queue should have the following methods:
 1. enqueue(item, priority): Add an item to the queue with a given priority.
 2. dequeue(): Remove and return the item with the highest priority (lowest priority number).

```
import heapq

class PriorityQueue:
    def __init__(self):
        self.elements = []

    def enqueue(self, item, priority):
        heapq.heappush(self.elements, (priority, item))

    def dequeue(self):
        if not self.is_empty():
            return heapq.heappop(self.elements)[1]
        raise IndexError("Dequeue from an empty priority queue")

    def is_empty(self):
        return len(self.elements) == 0

    def display(self):
        print("Priority Queue:")
        for priority, item in sorted(self.elements):
            print(f"Priority: {priority}, Item: {item}")

# Example usage:
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("Task 1", priority=2)
    pq.enqueue("Task 2", priority=1)
    pq.enqueue("Task 3", priority=3)

    pq.display()

    print("Dequeue:", pq.dequeue())
    print("Dequeue:", pq.dequeue())
    print("Dequeue:", pq.dequeue())
```

```
C:\Users\aelur\OneDrive\Desktop\Ai_assistanr>
Priority Queue:
Priority: 1, Item: Task 2
Priority: 2, Item: Task 1
Priority: 3, Item: Task 3
Dequeue: Task 2
Dequeue: Task 1
Dequeue: Task 3
```

Task 8 – Deque:

Use AI to implement a double-ended queue using collections.deque.

Prompt: Implement a double-ended queue (Deque) in Python using collections.deque. The class should allow insertion and removal of elements from both the front and rear. Include methods for inserting at front and rear, removing from front and rear, checking size, and verifying if the deque is empty.

```
from collections import deque
class DequeDS:
    def __init__(self):
        self.items = deque()

    def insert_front(self, item):
        self.items.appendleft(item)

    def insert_rear(self, item):
        self.items.append(item)

    def remove_front(self):
        if self.is_empty():
            return None
        return self.items.popleft()

    def remove_rear(self):
        if self.is_empty():
            return None
        return self.items.pop()

    def peek_front(self):
        if self.is_empty():
            return None
        return self.items[0]

    def peek_rear(self):
        if self.is_empty():
            return None
        return self.items[-1]

    def size(self):
        return len(self.items)

    def is_empty(self):
        return len(self.items) == 0

# Example usage:
if __name__ == "__main__":
    deque_ds = DequeDS()
    deque_ds.insert_rear(1)
    deque_ds.insert_rear(2)
    deque_ds.insert_front(0)
    print(deque_ds.peek_front())
    print(deque_ds.peek_rear())
    print(deque_ds.remove_front())
```

```
C:\Users\aelur\OneDrive\Desktop\Ai_assistant>C:/Pyt
0
2
0
2
1
```

Explanation:

- A deque (double-ended queue) allows insertion and deletion from both ends efficiently.
- Using `collections.deque` ensures all operations like `append`, `appendleft`, `pop`, and `popleft` run in $O(1)$ time.
- It is commonly used in sliding window problems and BFS algorithms.